PLICHARD Matthew
3A ICy

24/04/2026

TP

# TP mini Discord

Université Polytechnique HAUTS-DE-FRANCE
INSA INSTITUT NATIONAL DES SCIENCES APPLIQUÉES HAUTS-DE-FRANCE

# Sommaire :


I. Phase 1 - GenServer 3

1.1 Fiche à compléter 3

- Q1. 3
- Q2. 3
- Q3. 3

Exercice 1.2 Salon-Test 4

II. Phase 2 Supervision et Robustesse 4

- 2-4 5
- 2-5 5

III. Phase 3 - Sécurité et commandes 6

Exercice 3.1 Pseudos uniques 6

IV. Bonus 10

# I. Phase 1 - GenServer

## 1.1 Fiche à compléter

```
def handle_call({:rejoindre, pid}, _from, state) do
  Process.monitor(pid)
  new_state = %{state | clients: [pid | state.clients]}
  {:reply, :ok, new_state}
end

def handle_call({:quitter, pid}, _from, state) do
new_state =%{state | clients: List.delete(state.clients, pid)}
{:reply, :ok,new_state}
end

def handle_cast({:broadcast, msg}, state) do
Enum.each(state.clients, fn client_pid -> send(client_pid, {:message, msg}) end)
{:noreply, state}
end

def handle_info({:DOWN, _ref, :process, pid, _reason}, state) do
new_state = %{state | clients: List.delete(state.clients, pid)}
{:noreply, new_state}
end
```

*Figure 1 : Complétion du fichier salon.ex*

- **Q1.**

On utilise Process.monitor pour que si un client se déconnecte soudainement (crash) le serveur soit averti. Grâce à cela, si un client se déconnecte, le serveur n'enverra pas de message à un PID qui n'existe plus.

- **Q2.**

L'utilisation de handle_info, permet lors d'une déconnexion soudaine, que le Salon sache comment réagir, en effet, lors de la déconnexion, le serveur reçoit un message :DOWN, mais sans notre fonction, il ne sait pas quoi en faire, grâce à ce morceau de code, il retire l'utilisateur concerné de la liste des utilisateurs.

- **Q3.**

Handle_call est synchrone ce qui signifie que le client attend une réponse du Salon, alors que handle_cast est asynchrone il n'attend aucune réponse.

Le broadcast est un cast, car après l'envoie d'un message, le client n'attend pas d'accusé de réception pour que ce soit rapide et fluide, en effet, en call, le broadcast devrait attendre que chaque client reçoive le message avant de pouvoir effectuer une autre action, ce qui peut être long selon les connexions et le nombre d'utilisateurs.

## Exercice 1.2 Salon-Test

```
{:via, Registry, {Dscord.Registry, "generam"}} |> GenServer.whereis() |> IO.inspect(label: "PID du salon")
PID du salon: #PID<0.171.0>
#PID<0.171.0>
iex(7)> pid = GenServer.whereis({:via, Registry, {Dscord.Registry, "generam"}})
#PID<0.171.0>
iex(8)> Process.exit(pid,:kill)
true
iex(9)> {:via, Registry, {Dscord.Registry, "generam"}} |> GenServer.whereis() |> IO.inspect(label: "PID du salon")
PID du salon: #PID<0.174.0>
#PID<0.174.0>
```

*Figure 2 : Résultat du Test*

```
@LunascroZ → /workspaces/dscord/dscord (main) $ iex -S mix
** (ArgumentError) The module MiniDiscord.Salon was given as a child to a supervisor but it does not exist.
    (elixir 1.14.0) lib/supervisor.ex:701: Supervisor.init_child/1
    (elixir 1.14.0) lib/supervisor.ex:809: Supervisor.child_spec/2
    (elixir 1.14.0) lib/dynamic_supervisor.ex:400: DynamicSupervisor.start_child/2
iex(1)>
nil
iex(2)> DynamicSupervisor.start_child(Dscord.SalonSupervisor, {Dscord.Salon, "test"})
{:ok, #PID<0.162.0>}
iex(3)> Dscord.Salon.rejoindre("test", self())
:ok
iex(4)> Dscord.Salon.broadcast("test", "hello!")
:ok
iex(5)> flush
{:message, "hello!"}
:ok
```

*Figure 3 : Résultat connexion avec NetCat*

```
@LunascroZ → /workspaces/dscord (main) $ nc localhost 4040
Bienvenue sur Dscord!
Entre ton pseudo : Matthew
Salons disponibles : test
Rejoins un salon (ex: general) : general
Tu es dans #general — écris tes messages !
🐾 Matthew a rejoint #general
Hello Guys
[Matthew] Hello Guys
🐾 Benoit a rejoint #general
Avis tomdachi life ?
[Matthew] Avis tomdachi life ?
[Benoit] Sympatoche
```

*Figure 4 : Installation et test bore* 👍

# II. Phase 2 Supervision et Robustesse

```
@LunascroZ → /workspaces/dscord (main) $ curl -L https://github.com/ekzhang/bore/releases/download/v0.5.0/bore-v0.5.0-x86_64-unknown-linux
-musl.tar.gz | tar xz
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
  0     0    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--     0
100 2007k  100 2007k    0     0  6386k      0 --:--:-- --:--:-- --:--:-- 6386k
@LunascroZ → /workspaces/dscord (main) $ ./bore local 4040 --to bore.pub
2026-04-24T12:44:06.429222Z  INFO bore_cli::client: connected to server remote_port=35279
2026-04-24T12:44:06.429248Z  INFO bore_cli::client: listening at bore.pub:35279
```

*Figure 5 : Recherche de PID et kill*

- ## 2-4

Oui le salon redémarre automatiquement après le kill, cela est dû au fait qu'on utile un Supervisor pour démarrer des salons (enfants) par conséquence, ils les surveillent, et si l'un d'eux se ferment subitement, alors il le redémarre automatique.

- ## 2-5

La différence entre one for one et one for all est au redémarrage. En effet, en one for one, si un processus a un problème, alors celle-ci est redémarré mais uniquement celle qui a crash redémarre. En one for all, dès qu'il y a un problème sur UN processus, tout les autres redémarrent avec.

```
def start_link(name) do
  GenServer.start_link(__MODULE__, %{name: name, clients: [], historique: []},
    name: via(name))
end
```

*Figure 6 : Ajout de l'historique à Salon*

```
def handle_cast({:broadcast, msg}, state) do
Enum.each(state.clients, fn client_pid -> send(client_pid, {:message, msg}) end)
new_historique = [msg | state.historique] |> Enum.take(10)
{:noreply, %{state |historique: new_historique}}
end
```

*Figure 7 : Ajout de la synchronisation d'historique à chaque nouveau message*

```
def handle_call({:rejoindre, pid}, _from, state) do
  Process.monitor(pid)
  state.historique
  |> Enum.reverse()
  |> Enum.each(fn msg -> send(pid, {:message, "[Archives] #{msg}"}) end)
  new_state = %{state | clients: [pid | state.clients]}
  {:reply, :ok, new_state}
end
```

*Figure 8 : Envoie de l'historique à chaque personne qui rejoint le salon*

```
def lister do
  Registry.select(Dscord.Registry, [{{:"$1", :_, :_}, [], [:"$1"]}])
end
```

*Figure 9 : Fonction lister déjà présente*

```
@LunascroZ → /workspaces/dscord (main) $ nc localhost 4040
Bienvenue sur Dscord!
Entre ton pseudo : Michel
Salons disponibles : aucun (tu seras le premier !)
Rejoins un salon (ex: general) : general
Tu es dans #general — écris tes messages !
🚪 Michel a rejoint #general
1
[Michel] 1
2
[Michel] 2
3
[Michel] 3
4
[Michel] 4
5
[Michel] 5
7
[Michel] 7
6
[Michel] 6
8
[Michel] 8
9
[Michel] 9
10
[Michel] 10
11
[Michel] 11
🚪 Matthew a rejoint #general
```

*Figure 10 : Test historique*

```
@LunascroZ → /workspaces/dscord (main) $ nc localhost 4040
Bienvenue sur Dscord!
Entre ton pseudo : Matthew
Salons disponibles : general
Rejoins un salon (ex: general) : general
Tu es dans #general — écris tes messages !
[Archives] [Michel] 2
[Archives] [Michel] 3
[Archives] [Michel] 4
[Archives] [Michel] 5
[Archives] [Michel] 7
[Archives] [Michel] 6
[Archives] [Michel] 8
[Archives] [Michel] 9
[Archives] [Michel] 10
[Archives] [Michel] 11
🚪 Matthew a rejoint #general
```

*Figure 11 : Fonctionnel* 👍

# III. Phase 3 - Sécurité et commandes

## Exercice 3.1 Pseudos uniques

```elixir
def start(_type, _args) do
  :ets.new(:pseudos, [:named_table, :public, :set])
  children = [
    {Registry, keys: :unique, name: Dscord.Registry},
    {DynamicSupervisor, strategy: :one_for_one, name: Dscord.SalonSupervisor},
    Dscord.ChatServer,
    {Task.Supervisor, name: Dscord.TaskSupervisor}
  ]
  opts = [strategy: :one_for_one, name: Dscord.Supervisor]
  Supervisor.start_link(children, opts)
```

*Figure 12 : Ajout de la ligne de créations du tableau de pseudos*

On le met en premier pour que ce soit la première chose qui soit créé.

```elixir
  defp pseudo_disponible?(pseudo) do
    case :ets.lookup(:pseudos,pseudo) do
      [] ->
        true
      [_autres] ->
        false
end
end

defp reserver_pseudo(pseudo) do
:ets.insert(:pseudos, {pseudo, self()})
end


defp liberer_pseudo(pseudo) do
:ets.delete(:pseudos, pseudo)
end
```

*Figure 13 : Ajout des fonctions de pseudo*

```
defp choisir_pseudo(socket) do
  :gen_tcp.send(socket, "Entre ton pseudo : ")
  {:ok, pseudo} = :gen_tcp.recv(socket, 0)
  pseudo = String.trim(pseudo)
if pseudo_disponible?(pseudo) do
      reserver_pseudo(pseudo)
      pseudo
      else
        :gen_tcp.send(socket, "Désolé, ce pseudo est déjà pris...\r\n")
        choisir_pseudo(socket)
    end
end
```

```
def start(socket) do
  :gen_tcp.send(socket, "Bienvenue sur Dscord!\r\n")
  pseudo = choisir_pseudo(socket)


  :gen_tcp.send(socket, "Salons disponibles : #{salons_dispo()}\r\n")
  :gen_tcp.send(socket, "Rejoins un salon (ex: general) : ")
  {:ok, salon} = :gen_tcp.recv(socket, 0)
  salon = String.trim(salon)

  rejoindre_salon(socket, pseudo, salon)
end
```

*Figure 14 et 15 : Modification de la logique de gestion de client pour l'utilisation des pseudos*

```
defp gerer_commande(socket,pseudo,salon,commande) do
  case String.split(commande, " ", parts: 2) do
    ["/list"] ->
      salons = Dscord.Salon.lister()
      :gen_tcp.send(socket, "Salons disponibles : #{Enum.join(salons, ", ")}\r\n")
      salon

    ["/join",nouveau_salon] ->
      Dscord.Salon.quitter(salon, self())
      rejoindre_salon(socket, pseudo, nouveau_salon)
      nouveau_salon

    ["/quit"] ->
      liberer_pseudo(pseudo)
      :gen_tcp.close(socket)
      exit(:normal)
    _ ->
      :gen_tcp.send(socket, "Commande inconnue\r\n")
      salon

end
end
```

*Figure 16 : Ajout de la logique de commande*

```
defp loop(socket, pseudo, salon) do

  receive do
    {:message, msg} ->
      :gen_tcp.send(socket, msg)
  after 0 -> :ok
  end


  case :gen_tcp.recv(socket, 0, 100) do
    {:ok, msg} ->
      msg = String.trim(msg)


      nouveau_salon = if String.starts_with?(msg, "/") do
        gerer_commande(socket, pseudo, salon, msg)
      else
        Dscord.Salon.broadcast(salon, "#{pseudo}: #{msg}\r\n")
        salon
      end
```

*Figure 17 : Modification de la logique de loop en conséquence*

```
Bienvenue sur Dscord!
Entre ton pseudo : Matthew
Salons disponibles : aucun (tu seras le premier !)
Rejoins un salon (ex: general) : general
Tu es dans #general — écris tes messages !
🚪 Matthew a rejoint #general
🚪 Matt a rejoint #general
/list
Salons disponibles : general
/join autre
Tu es dans #autre — écris tes messages !
🚪 Matthew a rejoint #autre
/quit
```

```
Bienvenue sur Dscord!
Entre ton pseudo : Matthew
Désolé, ce pseudo est déjà pris...
Entre ton pseudo : Matt
Salons disponibles : general
Rejoins un salon (ex: general) : general
Tu es dans #general — écris tes messages !
[Archives] 🚪 Matthew a rejoint #general
🚪 Matt a rejoint #general
```

*Figure 18 et 19 : Test Concluant*

# IV. Bonus

```
def start_link(name) do
  GenServer.start_link(__MODULE__, %{name: name, clients: [], historique: [], password: nil},
    name: via(name))
end
```

```
  def handle_call({:set_password, pass}, _from, state) do
  {:reply, :ok, %{state | password: pass}}
end

  def handle_call({:check_password, pass_tente}, _from, state) do
  reponse = cond do
    state.password == nil -> :ok
    state.password == pass_tente -> :ok
    true -> :error
  end
  {:reply, reponse, state}
  end
```

```
def proteger(salon, password), do: GenServer.call(via(salon), {:set_password, password})
def verifier_password(salon, password), do: GenServer.call(via(salon), {:check_password, password})
```

*Figure 20,21,22: Ajout de fonction à Salon pour gérer les mots de passe*

```
  defp rejoindre_salon(socket, pseudo, salon) do
  Dscord.Salon.rejoindre(salon, self())
  Dscord.Salon.broadcast(salon, "📢 #{pseudo} a rejoint ##{salon}\r\n")
  :gen_tcp.send(socket, "Tu es dans ##{salon} — écris tes messages !\r\n")

  loop(socket, pseudo, salon)
end
```

```
  def start(socket) do
    :gen_tcp.send(socket, "Bienvenue sur Dscord!\r\n")
    pseudo = choisir_pseudo(socket)
    :gen_tcp.send(socket, "Salons disponibles : #{salons_dispo()}\r\n")
    premier_salon(socket,pseudo)
  end

  defp premier_salon(socket, pseudo) do
    :gen_tcp.send(socket, "Quel salon veux-tu rejoindre ? (ex: general) : ")
    {:ok, choix} = :gen_tcp.recv(socket, 0)
    choix = String.trim(choix)
    {nom, pass} = case String.split(choix, " ", parts: 2) do
    [n, p] -> {n, p}
    [n] -> {n, nil}
  end
  assurer_salon_existe(nom)
  case Dscord.Salon.verifier_password(nom, pass) do
      :ok ->
        rejoindre_salon(socket, pseudo, nom)
      :error ->
        :gen_tcp.send(socket, "Accès refusé : mot de passe incorrect.\r\n")
        premier_salon(socket, pseudo)
    end
  end


  defp assurer_salon_existe(salon) do
  case Registry.lookup(Dscord.Registry, salon) do
    [] ->
      DynamicSupervisor.start_child(
        Dscord.SalonSupervisor,
        {Dscord.Salon, salon})
    _ -> :ok
  end
end
```

*Figure 23 & 24 : Modification de la logique d'accession au salon*

```
defp gerer_commande(socket,pseudo,salon,commande) do
  case String.split(commande, " ", parts: 2) do
    ["/list"] ->
      salons = Dscord.Salon.lister()
      :gen_tcp.send(socket, "Salons disponibles : #{Enum.join(salons, ", ")}\r\n")
      salon

    ["/join",reste] ->
      {nom_salon, password} = case String.split(reste, " ", parts: 2) do
      [nom, pass] -> {nom, pass}
      [nom] -> {nom, nil}
   end
      case Dscord.Salon.verifier_password(nom_salon, password) do
        :ok ->
          Dscord.Salon.quitter(salon, self())
          rejoindre_salon(socket, pseudo, nom_salon)
        :error ->
          :gen_tcp.send(socket, " Mot de passe incorrect pour ##{nom_salon}\r\n")
          salon
      end

    ["/quit"] ->
      liberer_pseudo(pseudo)
      :gen_tcp.close(socket)
      exit(:normal)
    ["/protect",password] ->
      Dscord.Salon.proteger(salon, password)
      :gen_tcp.send(socket, " Le salon ##{salon} est maintenant protégé par mot de passe.\r\n")
      salon
    _ ->
      :gen_tcp.send(socket, "Commande inconnue\r\n")
      salon

end
end
```

*Figure 25 : Modification de la logique de la commande /join et ajout de /protect pour protéger un salon avec mot de passe*

On a ajouté un mot de passe au salon, quand on veut rejoindre un salon on a la possibilité d'ajouter un mot de passe en +. On test si le mot de passe fonctionne, si oui on rejoint le salon, si non on envoie un message erreur.

La commande /protect mdp permet de protéger un salon avec un mot de passe.

```
Bienvenue sur Dscord!
Entre ton pseudo : Matthew
Salons disponibles : aucun (tu seras le premier !)
Quel salon veux-tu rejoindre ? (ex: general) : general
Tu es dans #general — écris tes messages !
📢 Matthew a rejoint #general
/protect azerty
 Le salon #general est maintenant protégé par mot de passe.
📢 Poule a rejoint #general
```

```
Bienvenue sur Dscord!
Entre ton pseudo : Poule
Salons disponibles : general
Quel salon veux-tu rejoindre ? (ex: general) : general
Accès refusé : mot de passe incorrect.
Quel salon veux-tu rejoindre ? (ex: general) : general azerty
Tu es dans #general — écris tes messages !
[Archives] 📢 Matthew a rejoint #general
📢 Poule a rejoint #general
```

*Figure 26 : Test Finaux*